

Typed Stage Contracts for Knowledge Graph Question Answering: Traceable Reasoning, Bounded Repair, and Trajectory Supervision

Anonymous authors

Paper under double-blind review

Abstract

Combining large language models (LLMs) with knowledge graphs (KGs) provides a practical interface for natural-language reasoning over structured data, but executable KGQA systems typically expose retrieved evidence or a final program rather than the sequence of intermediate states that produced the answer. This makes it difficult to localise failures, revise them in a controlled manner, or reuse failed and revised reasoning as supervision. We introduce a KGQA framework that represents semantic interpretation, program construction, grounding, deterministic execution, and release as a persistent typed reasoning trajectory. Stage-specific contracts evaluate these states and return structured diagnostics for bounded repair; after inference, the retained trajectory also supports correctness-filtered supervision for direct plans, repairs, and preferences without exposing reference answers online. We instantiate the framework on a frozen UK procurement graph and a 12,828-question program-first benchmark, with additional tests of paraphrase robustness and cross-domain interface reuse. Formal repair recovers 48.18% of answerable attempts entering revision, while 18.28% of answer-producing teacher trajectories that satisfy all online contracts remain oracle-wrong, highlighting the distinction between runtime acceptance and offline correctness. Across three repeated inference runs on the fixed 2,025-question primary set, the complete local Qwen runtime reaches 85.68% accuracy, compared with 69.19% for the cloud runtime. Overall, the results show how persistent typed trajectories can provide a common interface for traceable execution, bounded repair, and supervision while keeping verification boundaries explicit.

1 Introduction

Large language models (LLMs) are effective at interpreting natural-language questions and combining information during reasoning, but their parametric knowledge and unconstrained generation make reliable grounding difficult (Toroghi et al., 2024; Luo et al., 2025; Agarwal et al., 2024). Knowledge graphs (KGs) complement these capabilities by providing explicit entities, relations, and schema over which reasoning can be grounded and executed. This has motivated KGQA systems that combine LLM-based interpretation with symbolic programs, grounded reasoning paths, or tool-based execution (Agarwal et al., 2024).

Grounding reasoning in a KG, however, does not by itself make the reasoning process traceable. Existing systems expose programs, grounded steps, reasoning paths, or execution state, but these objects remain tied to their respective reasoning mechanisms (Toroghi et al., 2024; Jiang et al., 2025; Luo et al., 2025). They do not provide a common record of how a question progresses from interpretation to program construction, grounding, execution, and release, or preserve the failed state that triggers revision. Consequently, an incorrect answer may not reveal where the error entered the computation, while successful execution does not establish that the intended question was represented correctly. We call the capability needed to address this gap *stage-level traceability*: a persistent typed record that links each reasoning state to its evidence, contract outcome, and revision context for subsequent repair and supervision.

We address this gap with a typed, stage-linked reasoning trajectory spanning semantic interpretation, program construction, grounding, deterministic execution, and release. Each transformation emits a retained state;

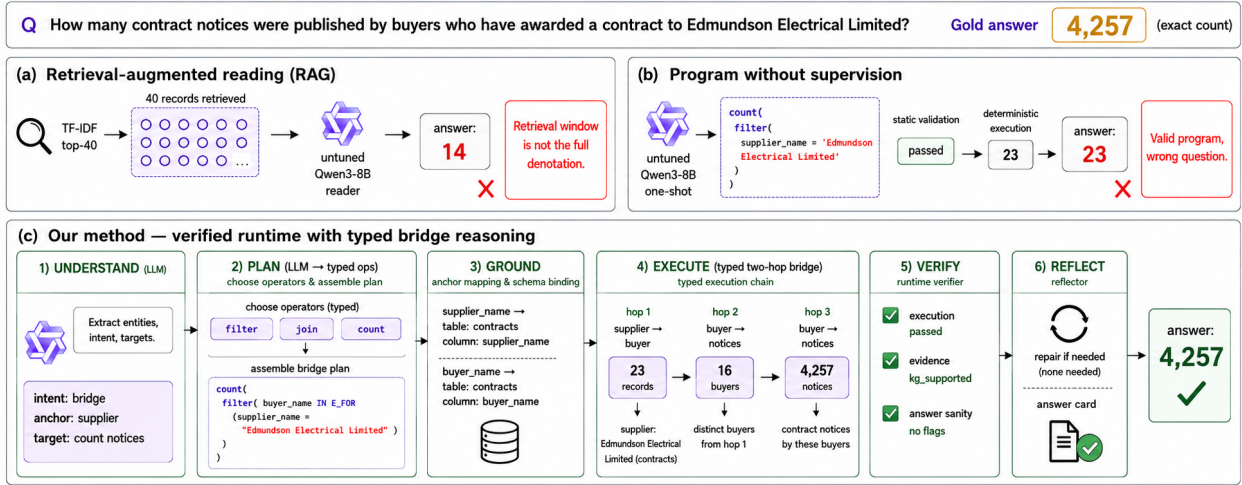


Figure 1: Running example. (a) Retrieval finds supplier records but not the full population required by the count. (b) A valid executable program counts those source records instead of traversing through their buyers. (c) The proposed runtime retains stage-linked provenance, constructs the required traversal, and checks each state before release.

stage-specific contracts decide whether computation continues, terminates, or enters bounded repair, while the same trajectory later supplies direct-plan, repair, and preference supervision after offline correctness filtering. Figure 1(c) illustrates the runtime, formalised in Sections 3 and 4. We instantiate it on a frozen UK public-procurement graph containing 215,221 contracts, with entity resolution, value semantics, and provenance rules fixed before inference (Open Contracting Partnership, 2020; Soylu et al., 2022), and a 12,828-question program-first benchmark spanning answerable, unsupported, ambiguous, and empty-denotation requests over lookup, aggregation, comparison, ranking, set, and multi-hop computations.

The evaluation is organised around three questions: whether stage-specific contracts expose failure states that support bounded repair; whether runtime acceptance is sufficient for selecting supervision; and whether the typed executable interface remains useful under surface and domain shift. Among answerable attempts entering formal repair, 48.18% finish with an oracle-correct result. Offline, 18.28% of answer-producing trajectories that satisfy all runtime contracts remain oracle-wrong, motivating a separate correctness filter before export. The verified records train Qwen and Llama models; on the primary test, the complete local Qwen runtime reaches 85.68% accuracy, raises release coverage from 82.20% to 91.36%, and reduces conditional error among released answers from 21.37% to 8.95% relative to the cloud runtime. PACS and WikiTableQuestions provide complementary tests of paraphrase robustness and cross-domain interface reuse.

This work makes three contributions. First, we introduce a typed reasoning trajectory that retains stage-linked computational provenance across interpretation, program construction, grounding, execution, and release, with deterministic contracts at each verification boundary. Second, we use the same persistent trajectory for bounded repair and for correctness-filtered supervision, linking online verification to later learning without exposing reference answers during inference. Third, we instantiate and evaluate the framework on a frozen procurement graph and program-first benchmark, including complete and hybrid runtimes, paraphrase robustness, and cross-domain interface reuse.

2 Related Work

Executable structured reasoning. LLM-based KGQA exposes graph structure through retrieval, reasoning paths, executable programs, and agentic interaction. StructGPT and RoG introduced structured interaction and relation-path reasoning, while SymKGQA and CodeAlignKGQA use executable symbolic or constrained-code representations (Jiang et al., 2023; Luo et al., 2024; Agarwal et al., 2024; 2025); later

systems combine graph exploration with agents or verifiable KG constraints (Jiang et al., 2025; Luo et al., 2025; Shan et al., 2026; Sui et al., 2025; Liu et al., 2026). Program-supervised benchmarks such as GrailQA and KQA Pro expose logical forms or compositional programs (Gu et al., 2021; Cao et al., 2022). Related semantic-parsing and text-to-SQL work maps language to logical forms, decomposes complex queries, and uses execution feedback for verification or refinement (Pasupat & Liang, 2015; 2016; Pourreza & Rafiei, 2023; Ni et al., 2023; Zhai et al., 2025; Zhang et al., 2025). These approaches make evidence or final computations inspectable, but do not necessarily retain one typed record linking interpretation, grounding, execution, and release; execution alone also cannot disambiguate logical forms with the same denotation (Zhong et al., 2020).

Feedback, repair, and supervision. Self-Refine, Reflexion, and CRITIC revise outputs using self-generated, episodic, or tool feedback (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024); process supervision and self-debugging provide related intermediate signals (Lightman et al., 2024; Chen et al., 2024). In KGQA, ARI-KBQA and DR2 add verification or diagnostic feedback (Gao et al., 2026; Liang et al., 2026), while execution feedback is also used to filter generated programs for training (Ni et al., 2023; Zhai et al., 2025; Zhang et al., 2025). These methods motivate revision and curation, but their published interfaces retain different parts of the process rather than one stage-aligned record spanning failure, verification, revision, and outcome. Our distinction is therefore not repair itself but its representation: one persistent typed trajectory spans interpretation, program construction, grounding, execution, and release and is retained for later supervision. Our runtime also separates online contract acceptance from offline typed-oracle filtering and represents unsupported, ambiguous, empty, and unreleased outcomes explicitly; the latter is related to selective QA (Kamath et al., 2020).

Contracts and provenance. Software contracts specify interface obligations, while runtime verification monitors declared properties during execution (Meyer, 1992; Leucker & Schallhart, 2009). Database provenance formalises links between outputs, source data, and transformations (Buneman et al., 2001). Our use of *computational provenance* is narrower: the trajectory records typed states, evidence, contract outcomes, and diagnostics between KGQA stages. The contracts cover declared KGQA conditions rather than proving semantic correctness.

3 Problem Formulation

Let K be a frozen knowledge-graph snapshot with schema $\mathcal{S}$ and q a natural-language question. Let $\mathcal{S}(q)$ denote the question-selected schema context. Rather than mapping q directly to an answer, we represent and retain the computation as explicit typed states: u records the interpreted request, z the executable computation, z_K its grounded form over K , and r the execution result and associated evidence.

The computation path uses a distinct operator for each transformation:

$$\begin{aligned} u &= M_u(q, \mathcal{S}(q)), & z &= \mathcal{B}(q, u, \mathcal{S}(q)), \\ z_K &= g_K(z, K, \mathcal{S}(q)), & r &= T_v(K, z_K), \\ \hat{y} &= h(r). \end{aligned} \tag{1}$$

Here M_u is the structured-understanding model, $\mathcal{B}$ the program builder that chooses deterministic compilation or learned planning, g_K the grounder, T_v the deterministic executor at version v , and h the output constructor. Verification is handled by a separate family of stage contracts. For state $s_i \in \{u, z, z_K, r, \hat{y}\}$,

$$\begin{aligned} (d_i, \delta_i) &= C_i(s_i, \mathcal{T}_{i-1}), & d_i &\in \{\text{CONTINUE, REJECT, REPAIR}\}, \\ \mathcal{T}_i &= \mathcal{T}_{i-1} \oplus (i, s_i, d_i, \delta_i), & \mathcal{T}_0 &= (q, \mathcal{S}(q)). \end{aligned} \tag{2}$$

Each C_i is a deterministic family of predicates over the current typed state and the evidence available in $\mathcal{T}_{i-1}$; it returns CONTINUE only when all applicable predicates hold. The contract neither constructs the next state nor changes the computation. A failed predicate produces δ_i , which names the violated condition, while $\oplus$ appends (i, s_i, d_i, δ_i) to the persistent trajectory. We use this stage-linked record as the computational

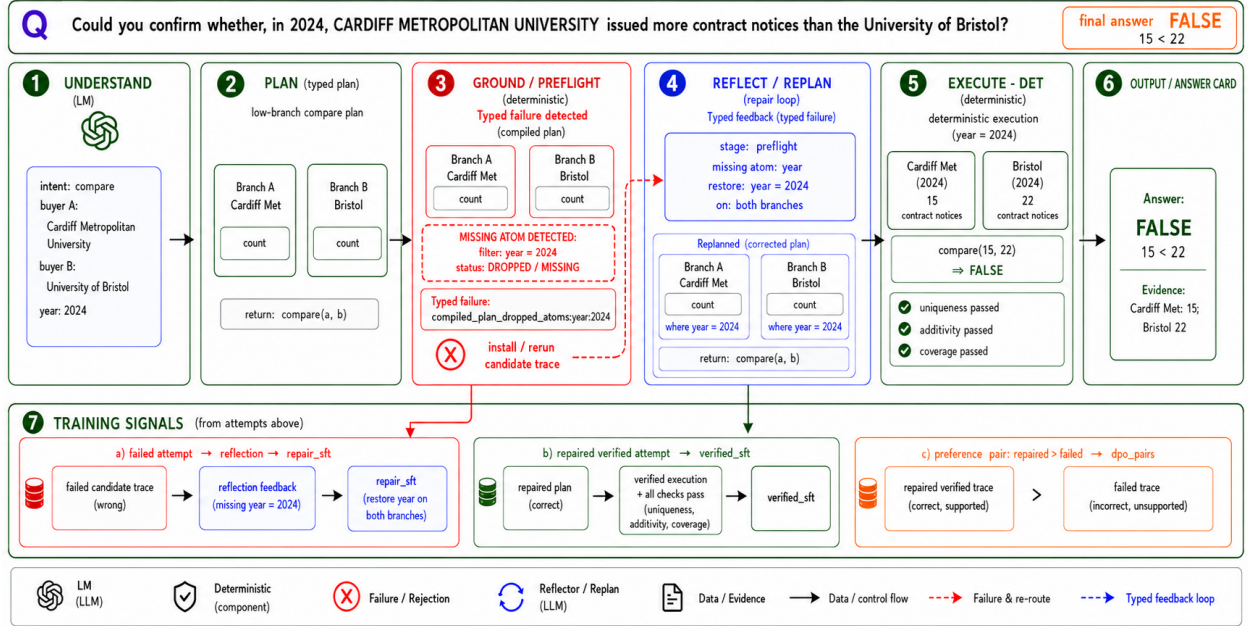


Figure 2: Online reasoning and offline supervision through one retained typed trajectory. During inference, contracts and bounded repair use only the question, graph, and retained runtime states; the reference answer is unavailable. After the run terminates, saved teacher trajectories may be evaluated by the offline typed oracle before supervision export.

provenance of the run: it preserves what each stage produced, how that state was judged, and which diagnostic governed the next transition. REJECT terminates the request and REPAIR invokes a bounded revision.

Repair acts on the computation rather than on the returned answer. Given the trajectory and diagnostic at the failed stage, the revision model produces a new interpretation, program, or both:

$$(u', z') = \mathcal{R}(q, \mathcal{T}_i, \delta_i), \quad z'_K = g_K(z', K, \mathcal{S}(q)), \quad r' = T_v(K, z'_K), \quad \hat{y}' = h(r'). \quad (3)$$

If only the executable computation is at fault, $u' = u$; if the interpretation is revised, the downstream computation is regenerated accordingly. Original and revised states remain in the same trajectory, so repair changes the computation without erasing the provenance of the failed attempt.

The reference answer y is unavailable to the model, contracts, and repair procedure during inference. Runtime decisions depend only on q , K , and the retained trajectory. Offline correctness is evaluated separately by comparing the returned prediction with y ; retained trajectories that satisfy the offline selection criteria can then be exported as supervision. These guarantees are intentionally distinct: static validity establishes that a program belongs to the closed grammar, executability that it runs on the backend, runtime release that the declared contracts pass, and oracle match that its denotation equals the benchmark reference. Only the last of these measures benchmark answer correctness, and none by itself establishes truth beyond the frozen corpus.

4 Method

4.1 System Overview

Figure 2 separates two authorities that operate at different times. *Online inference is oracle-free*: given q and schema context $\mathcal{S}(q)$, M_u produces structured interpretation u ; $\mathcal{B}$ constructs executable program z ; g_K

grounds it against the frozen graph; T_v executes it; and h constructs the candidate answer and evidence. Contracts use only the question, graph, and retained trajectory to continue, terminate, repair, or release the run. A repairable violation stores the failed state and diagnostic before bounded repair, which replays the affected downstream stages without access to the reference answer. *Offline curation begins only after the run has terminated*: the hidden typed oracle then scores saved teacher trajectories and decides whether they may be exported as supervision. Offline oracle decisions never feed back into the run that produced the trajectory.

The persistent trajectory is the common data structure across this boundary, not a shared decision authority. Online, its typed states and contract outcomes support traceability and revision. Offline, a completed trajectory becomes a curation object from which direct-plan, repair, and preference supervision can be constructed after oracle filtering. Sections 4.2–4.6 describe the online states and checks; Section 4.7 describes offline trajectory selection and export.

4.2 Frozen Graph and Schema Context

All reasoning is performed against a fixed graph snapshot K constructed from UK public procurement releases from 2022–2026. For each Open Contracting Identifier (OCID), the loader retains the latest parseable release (Open Contracting Partnership, 2020). Entity resolution and monetary-value semantics are fixed before inference: organisations are merged only under identifier or exact-match rules, while monetary values follow the precedence award, linked contract, then tender, with additivity recorded explicitly. The resulting graph contains 215,221 contract nodes and 131,502 canonical organisation nodes. Full construction rules, provenance, coverage, and graph statistics are reported in Appendix A.

For each question, a rule-based selector constructs the schema context $\mathcal{S}(q)$, exposing the relevant fields, relations, operator signatures, semantic roles, and value domains. It contains no result rows or reference answers, so it constrains program construction without providing the answer itself.

4.3 Structured Understanding

The first learned stage receives the question q and its schema context $\mathcal{S}(q)$, but no graph records, execution results, or reference answer. We abstract its structured interpretation as

$$u = M_u(q, \mathcal{S}(q)) = (\tau, L, R, P, D, A), \quad \phi_u(q, u) := [L(u) \subseteq L(q)] \wedge [C_q^{\text{mapped}} \subseteq C(u)], \quad (4)$$

where τ is the answer signature, L the explicit literals, R the role-bearing entity mentions, P the requested operations, D their dependencies, and A the answerability decision. ϕ_u is a target invariant supplied to the structural contract C_u , not an assumption that every generated interpretation satisfies it. Its two clauses ask whether extracted literals are supported by the question and whether conditions recognised by the schema mapper are retained. A mechanically detected violation produces a diagnostic; a passing check records that the mapped conditions represented by the contract are retained. Section 5.3 separately evaluates runtime acceptance against the offline answer oracle.

Representing structured understanding separately from program construction makes semantic commitments explicit before backend planning. The year, category, entity role, comparison direction, and requested answer object can therefore be checked before an executable program is chosen. The retained interpretation also separates a mismatch in what the question requires from a mismatch in how that requirement is realised as a computation, providing a precise state for later verification and repair. This separation requires an additional learned call relative to direct question-to-program generation while providing an explicit, validated interpretation state.

The answerability field A may mark a request as unsupported or ambiguous, but cannot declare a clean no-result case, since emptiness can only be established by execution against K . A deterministic validator additionally checks references, dependency order, the answer signature, and the required form of terminal non-answer decisions. The concrete Step-1 output schema, allowed literal slots, prompt, and validation logic are specified in Appendices B and C.

4.4 Executable Program Construction

The structured interpretation u specifies what the question requires but not yet the executable computation, so we construct $z = \mathcal{B}(q, u, \mathcal{S}(q))$. The program language is closed over the operator set supported by the procurement graph and benchmark, which limits general-purpose expressivity but gives every admitted computation a declared structure and type signature. The builder maps u to this common typed program interface; when learned program construction is required, the Step 2 graph planner receives q , u , $\mathcal{S}(q)$, and the allowed operator interface and proposes z . A further structural sample is considered only after a detectable failure; this program-construction retry is distinct from the feedback-conditioned formal repair below. Program construction never receives execution results or the reference answer. Exact builder and call interfaces are specified in Appendices C and B.

The executable representation is a recursive algebra tree,

$$z = (N, E, \rho), \quad \text{op}(n) \in \mathcal{O}, \quad \text{depth}(z) \leq 16, \quad |N| \leq 64, \quad (5)$$

where N is the set of computation nodes, E records their dependencies, ρ is the root operation producing the returned value, and $\mathcal{O}$ is the fixed operator set. The depth and node limits are hard runtime guards rather than benchmark-design parameters; they prevent pathological over-composition and bound the programs presented to validation and execution.

The operator inventory reflects the structure of the procurement graph and the question families evaluated in this study. Nodes operate over *Records*, *Values*, *Groups*, numbers, Booleans, and scalar values. Filtering and projection express field and entity constraints; aggregation supports counts and monetary totals; grouping and ranking support comparative procurement questions; and comparison, set operations, and computed membership support relational and multi-hop queries. The language therefore contains the operations required by the supported schema while excluding unconstrained model-defined operations. Full operator signatures and program bounds are given in Appendix D.

Relational queries are expressed by composing these operations rather than leaving intermediate hops implicit. For the running example in Figure 1, the required bridge can be written schematically as

$$z = \text{count}(\text{filter}[\text{buyer} \in \text{values}_{\text{buyer}}(\text{filter}[\text{supplier} = s])]). \quad (6)$$

The inner filter identifies records associated with supplier s , the projection materialises the corresponding buyer set, and the outer filter constructs the target population before the final count. The intermediate set is therefore part of the executable state rather than an implicit reasoning step and can be checked independently.

Before grounding, a deterministic validator checks both structural validity and consistency with u . Its checks follow the same domain contract as the operator set: operator signatures and dependencies must form a valid computation, while years, procurement categories, CPV codes, titles, numeric literals, buyer and supplier roles, comparison direction, and return type must remain consistent with the question and schema. These checks prevent a structurally valid program from silently changing procurement conditions represented by the checker. Programs containing unsupported operators, invalid compositions, mechanically detectable omissions, or incompatible return types are rejected before grounding, so the retained program state has a declared structural and semantic interface before graph binding.

4.5 Grounding and Deterministic Execution

Although z is structurally executable, it still contains symbolic schema fields, entity mentions, and literals. Grounding resolves them against K and $\mathcal{S}(q)$ to produce z_K , normalising types, resolving buyer and supplier mentions, and applying domain constraints such as schema membership, exhaustive population selection, and the monetary additivity guard. Ambiguous references fail explicitly rather than being chosen for executability. z_K is then evaluated over the frozen record universe with fixed operator semantics: filters construct populations, projections produce distinct values, grouping forms keyed populations, counts deduplicate contract identifiers, sums use decimal accumulation, and ranking ties follow a stable order. The runtime retains grounded constraints, execution status, output size, and bounded evidence; detailed grounding, operator, and evidence rules are in Appendices C and D.

Grounding and execution can therefore be written as

$$z_K = g_K(z, K, \mathcal{S}(q)), \quad r = T_v(K, z_K), \quad T_v^{(t_1)}(K, z_K) = T_v^{(t_2)}(K, z_K), \quad (7)$$

for fixed K , z_K , and executor version v . Fixed grounding rules and operator semantics reduce runtime flexibility but make the computation reproducible and keep the grounded program, execution state, and evidence linked in the retained trajectory. Deterministic replay does not establish that z faithfully represents the question; semantic consistency is checked at the stage boundaries described next.

4.6 Stage-Specific Verification and Bounded Repair

Each contract C_i is evaluated when its required evidence becomes available. The structural contract is used at both understanding and program construction, while grounding, execution, and release use separate classes. For example, grounding requires referenced fields to resolve to the declared schema and entity bindings to be unambiguous. A repairable violation stores diagnostic δ_i with the failed state, making the first detectable boundary explicit; failures outside the encoded predicates remain undetected. A passing stage advances, whereas a repairable failure appends its state and diagnostic to the trajectory. Repair receives the question, retained interpretation, program, and failure record—but not the expected answer—then revises the computation and replays downstream stages. Unsupported, ambiguous, and clean empty outcomes are terminal. Full predicates, diagnostics, repair eligibility, and implementation details appear in Appendix C and Appendix B.

Repair is bounded to one feedback revision during main evaluation and two during teacher collection, which fixes runtime cost but leaves some recoverable failures unresolved. The repair audit includes only this feedback-conditioned route, not structural resampling during program construction. The retained trajectory also connects online verification to later supervision: first-pass computations yield direct-plan targets, successful repairs preserve failed state, diagnostic, and revision, and accepted–rejected alternatives form preference pairs. These records remain candidates until the offline correctness filter in Section 4.7; runtime acceptance and offline answer correctness remain separate authorities, measured in Section 5.3.

Passing every declared contract means only that no represented violation remains. A program may still execute and release while omitting or misrepresenting a condition that is not captured by the encoded checks. Section 5.3 measures this residual gap against the offline typed oracle.

4.7 Benchmark Construction and Trajectory Supervision

The procurement benchmark is constructed from the frozen graph in a program-first manner. It is designed to exercise the procurement-specific schema, operator semantics, and verification boundaries of the deployed runtime rather than to replace general-purpose KGQA benchmarks. The generator first creates 10,000 executable answer specifications, which are executed and checked before natural-language generation. A second consistency screen verifies that each generated question preserves the filters and operation of its underlying computation, leaving 9,044 accepted *base* question–program pairs. The next stage is deliberately non-one-to-one: answerability variants and additional surface realisations create derived rows from a base executable program, after which whole-program grouping and class balancing determine the retained corpus and split composition. These operations add 3,784 derived rows, producing the final 12,828-question benchmark. All rows derived from the same executable program are kept within one split, preventing paraphrases or answerability variants from crossing partitions. A second implementation recomputes answerable cases as a cross-check under the same graph and data conventions. Appendix E gives the complete construction and integrity audit.

The full runtime is applied to the 9,267-question training partition. Before supervision export, answer-producing trajectories pass output-shape and hidden typed-oracle checks in addition to the online runtime contracts. The oracle is used only for offline selection and removed before export. Retained trajectories provide direct-plan and repair targets, while accepted–rejected alternatives provide preference supervision. Export statistics and information boundaries are reported in Appendices F and B.

Reuse of the executable interface is further tested on PACS and WikiTableQuestions (WTQ) (Pasupat & Liang, 2015). PACS contains 922 evaluation items constructed from executable typed trees spanning seven operation families and three complexity levels, with execution-agreement and overlap checks applied before evaluation. WTQ uses 4,344 questions over 421 held-out tables, represented through typed views for deterministic execution and raw views for official scoring. These experiments reuse the recursive algebra, validator, and executor rather than the full procurement runtime. WTQ adds one row-preserving arg-extremum node, `extreme_rows`, while retaining the same typed execution pattern. Full protocols are given in Appendices G and H.

4.8 Evaluation Protocol

Training follows a teacher-to-student sequence. The cloud runtime uses `gpt-5.4-nano` for Step 1 and `grok-4-1-fast-non-reasoning` for Step 2, both at temperature zero. The cloud teacher collects candidate trajectories from the 9,267-question training partition. Runtime checks are applied during execution, followed by offline correctness filtering before any trajectory is exported for training. The retained trajectories provide supervision for structured understanding, direct program construction, and repair conditioned on a structured diagnostic. The resulting local models are then run on the training questions to generate model-specific on-policy trajectories for RSFT, while accepted–rejected alternatives from the teacher and on-policy runs provide preference supervision for DPO. Exact training-set construction and export counts are reported in Appendix F. Step 1 and Step 2 SFT use three epochs at 10^{-4} ; Step 2 RSFT uses two epochs at 5×10^{-5} , and DPO uses one epoch at 5×10^{-6} with $\beta = 0.1$. All local checkpoints use 4-bit QLoRA with rank 64, $\alpha = 128$, dropout 0.05, bfloat16 computation, effective batch size 16, a 6,144-token cutoff, and cosine decay. Full optimisation and checkpoint configurations are given in Table 10.

Before primary evaluation, a balanced 260-question development subset is held out from the 2,285-question frozen evaluation partition, leaving a disjoint 2,025-question primary set. Step 2 checkpoints are compared within each model family, and the selected planner is paired with the corresponding Step 1 SFT model to form the complete local runtime. The cloud runtime is evaluated three times on the same development subset, for which we report the mean and standard deviation. After checkpoint selection, the frozen cloud, local, and hybrid configurations are each evaluated three times on the same primary questions. These repeated runs measure inference variability of a fixed system rather than training-seed or test-sample variation. Temperature-zero decoding does not imply bitwise-deterministic serving, so the repeats quantify serving-time variability. Headline primary accuracies are run means, with standard deviations below two percentage points. We report overall correctness together with answer coverage and conditional error on answerable questions, correct rejection of ambiguous or unsupported questions, and correctness on valid empty denotations. All saved predictions are rescored with the same strict typed comparator. The primary system comparison evaluates cloud, local, and hybrid runtimes over the same frozen graph, grounding rules, deterministic executor, runtime contracts, repair budget, and scorer. Complete cloud and local runtimes differ in the learned Step 1 and Step 2 components, while hybrid runtimes retain cloud Step 1 structured understanding and substitute the selected local Step 2 planner. The comparison therefore supports system-level conclusions; the hybrid configurations provide a separate stage-wise substitution analysis.

The access comparison instead holds the answering model fixed while varying the available evidence. Closed-book answering uses parametric knowledge only; retrieval reading uses TF-IDF top- k records; and gold-constraint retrieval constructs the retrieval query from the reference program’s constraint literals. The latter removes query formulation as a source of error and tests whether bounded retrieved evidence is sufficient when the intended constraints are already known. Appendix I summarises the decoding, repair, and oracle-access boundaries for each evaluation protocol, while Appendix J records the implementation entry points and artifact ledger used to reproduce the reported results.

5 Results

For evaluation, *answerable* denotes the 1,725 primary items with value-bearing targets. The remaining 300 cases comprise 200 unsupported or ambiguous requests and 100 executable empty-denotation queries, whose correct outcome is a terminal no-result status.

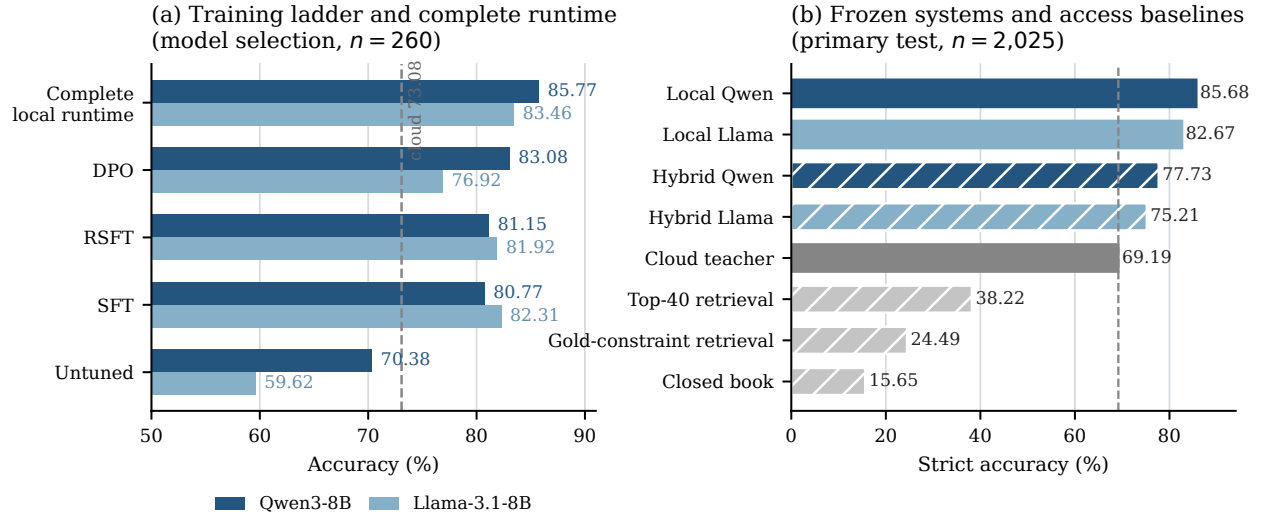


Figure 3: Model selection and primary evaluation. (a) Development checkpoints ($n = 260$); dashed line: three-run cloud mean (73.08%, SD 1.1 pp). (b) Primary test ($n = 2,025$); all displayed systems are means over three repeated inference runs on the same frozen questions. Run-to-run SDs (pp) are: cloud 0.96, local Qwen 1.38, local Llama 0.51, hybrid Qwen 1.17, hybrid Llama 0.56, top-40 retrieval 0.58, gold-constraint retrieval 0.61, and closed book 0.74.

5.1 Model Selection and End-to-End Performance

We use the cloud teacher as the reference system. On the 260-question development set, the complete cloud runtime averages 73.08% over three runs, with a standard deviation of 1.1 percentage points. The same teacher collects trajectories from the 9,267-question training partition. After offline filtering, these records supervise Qwen3-8B and Llama-3.1-8B; the resulting local models then generate model-specific trajectories for RSFT and accepted-rejected pairs for DPO, as described in Section 4.8. Figure 3(a) summarises checkpoint selection. Initial Step 2 SFT raises Qwen from 70.38% to 80.77% and Llama from 59.62% to 82.31%. Later optimisation is model-dependent: DPO improves Qwen but reduces Llama, despite Llama slightly exceeding Qwen after SFT. Similar DPO sensitivity has been reported in structured reasoning (Liu et al., 2025); we therefore select the best checkpoint within each model family. Pairing the selected Step 2 planner with the corresponding Step 1 SFT model yields complete Qwen and Llama runtimes of 85.77% and 83.46%, respectively, both above the cloud mean. Since this set is used for checkpoint selection, these results do not constitute an independent generalisation estimate.

Figure 3(b) evaluates the frozen configurations on the primary test. Across three repeated inference runs on the same questions, the complete local Qwen and Llama runtimes average 85.68% (SD 1.38 pp) and 82.67% (SD 0.51 pp), compared with 69.19% (SD 0.96 pp) for the cloud runtime. The hybrid Qwen and Llama configurations average 77.73% (SD 1.17 pp) and 75.21% (SD 0.56 pp), respectively. The 16.49-point mean gap between the complete local Qwen and cloud runtimes is therefore substantially larger than the observed run-to-run variability of either configuration; statistical comparison is based separately on the paired item-level predictions below. Hybrid runtimes retain cloud Step 1 structured understanding and replace Step 2 with the selected local planner, reaching 77.73% with Qwen and 75.21% with Llama. This is a stage-wise system comparison, not a strict Step 2 ablation: the archived serving configurations are not fully matched, although the downstream deterministic runtime is fixed. Accordingly, the 85.68% result is attributed to the complete local Qwen runtime; the hybrid rows are not used to assign that gain causally to Step 2. Because the local models receive in-domain trajectory supervision whereas the cloud runtime does not, this comparison measures complete-system performance rather than the isolated effect of the trajectory representation. The access controls are substantially lower: top-40 retrieval reaches 38.22% (SD 0.58 pp), gold-constraint retrieval 24.49% (SD 0.61 pp), and closed-book answering 15.65% (SD 0.74 pp). Even with

intended constraints supplied, gold-constraint retrieval remains weak when the required population must be recovered from bounded evidence rather than enumerated over the graph. The accuracy values above are three-run means; the coverage, release, and paired-transition statistics below use the archived local-Qwen run retained for item-level analysis and are therefore not averaged across runs. On the 1,725 answerable questions, local Qwen raises release coverage from 82.20% to 91.36% and reduces conditional error among released answers from 21.37% to 8.95% relative to the cloud runtime; correct rejection improves from 188/200 to 200/200 and valid empty outcomes from 98/100 to 100/100, so the gain is not obtained by withholding more answers. On the same archived run, local Qwen is correct on 1,435/1,725 answerable questions (83.19%).

On the archived paired prediction set, local Qwen gains 371 cases over the cloud runtime and loses 37 in the reverse direction. The paired difference is significant under the exact McNemar test (McNemar, 1947) ($p = 1.78 \times 10^{-70}$). Because the graph, deterministic backend, contracts, repair budget, and scorer are fixed, this establishes a systematic system-level difference between the frozen configurations rather than attributing the gain to any single learned stage. Appendix K summarises the held-fixed and changed components for this and the later robustness/reuse comparisons.

5.2 Stage-Level Traceability and Bounded Repair

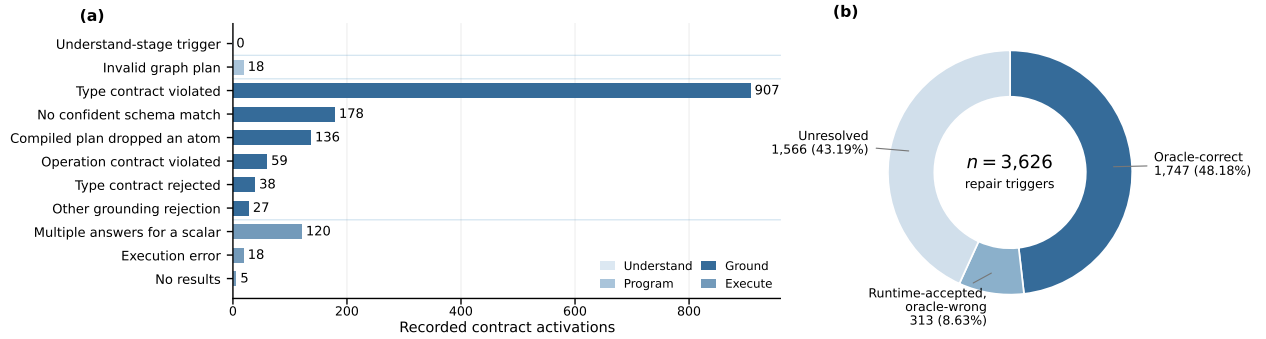


Figure 4: Stage-specific contract activations and bounded repair. (a) Runtime activations during teacher collection by firing stage. (b) Outcomes for 3,626 answerable attempts entering formal repair.

Figure 4(a) reports recorded runtime-contract violations: 18 during program construction, 1,345 during grounding, and 143 during execution. The understanding contract is evaluated but has no represented violation in this audit; most semantic inconsistencies become mechanically decidable only after grounding. Thus 1,363/1,506 (90.5%) violations occur before execution. Panel (a) counts predicate violations, whereas panel (b) counts answerable attempts entering formal repair, so their totals are not expected to match. The ledger distinguishes unresolved schema matches, dropped constraints, and operation- or type-contract violations rather than collapsing them into final-answer failure. These records also preserve terminal semantics: repairable violations enter bounded revision, unsupported or ambiguous requests terminate as rejection, clean empty denotations return no-result, and exhausted answerable requests remain unreleased. On the primary test, local Qwen correctly handles all 200 unsupported/ambiguous and 100 empty-denotation cases, while 149 answerable cases remain unreleased. Figure 4(b) tests whether the recorded failure state can support revision. Of 3,626 answerable attempts entering bounded repair, 1,747 are oracle-correct after repair (48.18%), 313 pass the runtime contracts but remain oracle-wrong (8.63%), and 1,566 remain unresolved (43.19%). The starting point is a diagnosed runtime failure with no releasable answer, so 48.18% measures recovery under feedback-conditioned repair with the declared budget, not the incremental benefit of typed diagnostics over blind retry. The unresolved category records the state after that fixed budget is exhausted; because no additional rounds are attempted, it does not distinguish cases that would recover with more revisions from cases limited by model or representation capacity. The qualitative cases in Appendix L illustrate both a type/return mismatch that is corrected by typed feedback and a multiple-answer failure that exhausts the teacher-harvest repair budget. Among the 1,747 oracle-correct repairs, the retained records are categorised as 1,604 plan-compilation, 125 executor-check, 17 schema-validation, and 1 graph-execution case.

5.3 Offline Correctness Filtering for Trajectory Supervision

The retained trajectory becomes a curation object after inference. Of 6,860 answer-producing teacher trajectories passing all runtime contracts, 5,606 agree with the typed oracle and 1,254 do not, so 18.28% would receive an incorrect positive label if execution success alone determined export, so the oracle is applied only offline. This establishes the need for correctness filtering of positive labels, not its marginal downstream gain. The 5,606 verified direct-plan trajectories form the eligible pool. A fixed bucket-stratified down-selection prevents high-frequency question clusters from dominating Step 2 SFT and defines the stored 2,787-row training and 57-row validation export. Section 5.2 adds 1,747 oracle-correct repaired trajectories and 390 accepted-rejected pairs; reference answers are removed from exported inputs and targets, with full counts in Appendix F. The retained provenance supports different learning objectives without changing the online interface. Direct-plan examples pair the question and interpretation with an accepted executable program; repair examples preserve the failed state, diagnostic, and revised program; preference pairs retain accepted and rejected alternatives under the same question and briefing. Figure 3(a) shows that this supervision is usable by both local model families: initial SFT improves Qwen and Llama, followed by model-dependent on-policy and preference refinement. The trajectory therefore links verification, repair, and learning through one typed record while keeping reference answers outside the online runtime.

Table 1: Single-call typed-algebra results. PACS uses the v1.1 evaluation manifest and deterministic re-execution of saved trees; WTQ uses the official evaluator. Scores across the two panels are not directly comparable.

Evaluation	Planner / supervision	Correct	Accuracy (%)
PACS A	Untuned Qwen3-8B	312 / 922	33.84
PACS A	Fast cloud planner	470 / 922	50.98
PACS A	Trained Qwen planner	722 / 922	78.31
PACS B	Trained Qwen planner	711 / 922	77.11
WTQ	Untuned Qwen3-8B	978 / 4,344	22.51
WTQ	Procurement-trained Qwen	1,187 / 4,344	27.33
WTQ	Answer-only self-harvest (A)	1,930 / 4,344	44.43
WTQ	Translated gold programs (C)	2,250 / 4,344	51.80

5.4 Robustness and Cross-Domain Interface Reuse

We test the executable interface under two complementary changes: surface paraphrase in PACS and cross-domain reuse on WTQ. On PACS, the trained Qwen planner reaches 78.31% on channel A and 77.11% on its paraphrased counterpart, versus 33.84% for untuned Qwen3-8B and 50.98% for the cloud planner. On WTQ, procurement-trained Qwen improves from 22.51% to 27.33% without WTQ-specific training, then reaches 44.43% and 51.80% with target-domain supervision. Published WTQ systems report 57.5% for TAPEX and 64.6% for Binder (Liu et al., 2022; Cheng et al., 2023), but these are contextual anchors rather than matched baselines: their supervision and inference protocols differ from our single typed-algebra generation and official rescoring. Across PACS and WTQ, the core recursive algebra, validator, and deterministic executor are reused; WTQ adds the row-preserving `extreme_rows` operator for arg-extremum cases. Interpretation, contracts, and grounding remain domain-specific. The WTQ result therefore tests architectural reuse of the executable core rather than zero-cost portability: adapting a new domain still requires a target schema together with its grounding and contract semantics. Full protocols are in Appendices G and H.

6 Discussion

Implications. The central contribution is representational: interpretation, program construction, grounding, execution, contract outcomes, and diagnostics become persistent runtime objects in one trajectory. The access-control results show why this matters computationally. Top-40 and gold-constraint retrieval remain far below the complete runtime, including when intended constraints are supplied, because aggregation and

multi-hop questions may require enumerating a population rather than reading a bounded evidence set. The supervision analysis exposes a different boundary: executability is not correctness. Among answer-producing teacher trajectories, 18.28% pass the online runtime contracts yet disagree with the offline oracle, making contract acceptance useful for controlling execution but insufficient as a supervision label. Repair completes the picture: 48.18% of answerable attempts entering formal revision recover to an oracle-correct result under the declared budget, while unresolved and runtime-accepted/oracle-wrong outcomes remain explicit in the retained record. The trajectory therefore serves as an interface for locating, revising, and curating computation without turning runtime verification into a claim of semantic completeness. This also distinguishes the trajectory from ordinary logging or a final executable program. Each retained state is typed and evaluated at an explicit stage boundary, so a failure is attached to the object and predicate that rejected it rather than only to a textual trace. When repair occurs, the failed state, diagnostic, revised proposal, and terminal outcome remain linked in the same record; when no repair succeeds, the unresolved boundary is preserved instead of being overwritten by the last model response. The same structure can then be inspected offline without changing the decision that was made online. A final program preserves the computation that was ultimately attempted, but not the rejected predecessor or the boundary that triggered revision. This distinction is what makes the trajectory useful both for diagnosis at inference time and for selecting supervision after the run has ended. PACS and WTQ probe whether that interface remains useful beyond the benchmark surface on which it was developed. PACS changes surface form while holding the executable tree and oracle fixed; WTQ changes domain, schema, and data source under its official scorer. The resulting gains support reuse of the typed executable core beyond one procurement surface generator, while the WTQ-specific grounding, contracts, and `extreme_rows` extension make the remaining adaptation cost explicit.

Limitations and Future work. Traceability is bounded by the states and predicates represented by the runtime: passing all declared contracts means that no represented violation remains, not that every semantic condition has been captured. Primary evaluation repeats each frozen configuration three times, capturing inference variability on a fixed test set; the checkpoints themselves are not retrained across random seeds, so training-seed variance is not estimated. The hybrid serving configurations support stage-wise system comparison rather than strict causal attribution, and the fixed repair budget does not identify which unresolved cases would recover under additional rounds. Likewise, WTQ demonstrates reuse of the executable core, but target-domain interpretation, grounding, and contract semantics still require domain adaptation. The repair audit has no blind-retry control, and the trajectory-learning results do not isolate offline correctness filtering from other fine-tuning effects. The procurement graph contains public, time-bounded records about real organisations, so outputs are scoped to the frozen 2022–2026 snapshot and practical use should preserve snapshot date and source provenance. Matched blind-retry, repair-budget, and execution-filtered controls can quantify the incremental effects of stage-specific diagnostics, additional repair rounds, and offline correctness filtering. A stratified human fidelity audit can provide an independent check of question–program semantic alignment beyond the automated consistency screen. The retained contract outcomes also suggest a natural process-supervision interface for reinforcement learning: stage-specific passes can provide intermediate positive signals, while diagnostics such as type-contract or execution violations can supply local negative feedback before the terminal answer is scored. Testing such dense feedback, together with unseen operator compositions, is a direct extension of the present trajectory design.

7 Conclusion

This work introduces typed stage contracts for traceable KGQA. The runtime retains interpretation, program construction, grounding, execution, release state, contract outcomes, and diagnostics as one stage-linked trajectory. This record localises recognised failures, supports bounded repair without reference-answer access, and provides structured supervision. Of answerable attempts entering formal repair, 48.18% finish oracle-correct; offline filtering separately removes runtime-accepted but oracle-wrong trajectories before export. The complete local Qwen runtime reaches 85.68% accuracy while improving release coverage and conditional error relative to the cloud runtime. PACS and WTQ further support reuse of the executable core across paraphrased surfaces and a different structured domain.

References

- Armin Toroghi, Willis Guo, Mohammad Mahdi Abdollah Pour, and Scott Sanner. Right for right reasons: Large language models for verifiable commonsense knowledge graph question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 6601–6633, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.378.
- Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 41540–41565. PMLR, 2025.
- Perna Agarwal, Nishant Kumar, and Srikanta Bedathur. SymKGQA: Few-shot knowledge graph question answering via symbolic program generation and execution. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10119–10140, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.545.
- Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9505–9523, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.468.
- Open Contracting Partnership. Open contracting data standard. Version 1.1.5, <https://standard.open-contracting.org/latest/en/>, 2020.
- Ahmet Soylu, Oscar Corcho, Brian Elvesæter, Carlos Badenes-Olmedo, Tom Blount, Francisco Yedro Martínez, Matej Kovacic, Matej Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Till C. Lech, and Dumitru Roman. TheyBuyForYou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022. doi: 10.3233/SW-210442.
- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9237–9251, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.574.
- Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024.
- Perna Agarwal, Nishant Kumar, and Srikanta Bedathur. Aligning complex knowledge graph question answering as knowledge-aware constrained code generation. In *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 3952–3978, Abu Dhabi, UAE, 2025. Association for Computational Linguistics.
- Yongxue Shan, Jie Peng, Zixuan Dong, Fei Hu, and Xiaodong Wang. Reasoning with ontology graph: Toward type-constrained knowledge graph question answering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10684–10697, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.489.
- Yuan Sui, Yufei He, Nian Liu, Xiaoxin He, Kun Wang, and Bryan Hooi. FiDeLiS: Faithful reasoning in large language models for knowledge graph question answering. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 8315–8330, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.436.
- Chenhui Liu, Jianpeng Zhou, and Jiahai Wang. Chain-of-Relations: Faithful and efficient LLM reasoning over knowledge graphs via relation-centric exploration. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 43108–43119, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.2138.

- Yu Gu, Sue Kase, Michelle T. Vanni, Brian M. Sadler, Percy Liang, Xifeng Yan, and Yu Su. Beyond I.I.D.: Three levels of generalization for question answering on knowledge bases. In *Proceedings of the Web Conference 2021*, pp. 3477–3488. Association for Computing Machinery, 2021. doi: 10.1145/3442381.3449992.
- Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, and Hanwang Zhang. KQA Pro: A dataset with explicit compositional programs for complex question answering over knowledge base. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6101–6119, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.422.
- Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 1470–1480, Beijing, China, 2015. Association for Computational Linguistics. doi: 10.3115/v1/P15-1142.
- Panupong Pasupat and Percy Liang. Inferring logical forms from denotations. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23–32, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1003.
- Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems*, volume 36, pp. 36339–36348, 2023. doi: 10.52202/075280-1577.
- Ansong Ni, Sridi Iyer, Dragomir Radev, Veselin Stoyanov, Wen-Tau Yih, Sida Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 26106–26128. PMLR, 2023.
- Bohan Zhai, Canwen Xu, Yuxiong He, and Zhewei Yao. Optimizing reasoning for text-to-SQL with execution feedback. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19206–19218, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.982.
- Jipeng Zhang, Haolin Yang, Kehao Miao, Ruiyuan Zhang, Renjie Pi, Jiahui Gao, and Xiaofang Zhou. ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 24305–24326, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.1320.
- Ruiqi Zhong, Tao Yu, and Dan Klein. Semantic evaluation for text-to-SQL with distilled test suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 396–411, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.29.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46534–46594, 2023. doi: 10.52202/075280-2019.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 8634–8652, 2023. doi: 10.52202/075280-0377.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.

- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to Self-Debug. In *International Conference on Learning Representations*, 2024.
- Jianqi Gao, Hang Yu, Jian Cao, Ranran Bu, Jinghua Tang, Nengjun Zhu, and Yonggang Zhang. Generating then refining for reliable knowledge base question answering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 32070–32087, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.1479.
- Gewen Liang, Mufan Xu, Kehai Chen, Wei Wang, Yuwei Wang, Muyun Yang, Tiejun Zhao, and Min Zhang. Diagnosing and remedying representation deficiencies for deterministic reasoning in KGQA. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 45036–45054, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.2086.
- Amita Kamath, Robin Jia, and Percy Liang. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 5684–5696, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.503.
- Bertrand Meyer. Applying “design by contract”. *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.
- Martin Leucker and Christian Schallhart. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. doi: 10.1016/j.jlap.2008.08.004.
- Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pp. 316–330. Springer, 2001. doi: 10.1007/3-540-44503-X_20.
- Hanbing Liu, Haoyang Li, Xiaokang Zhang, Ruotong Chen, Haiyong Xu, Tian Tian, Qi Qi, and Jing Zhang. Uncovering the impact of chain-of-thought reasoning for direct preference optimization: Lessons from text-to-SQL. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 21223–21261, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1031.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations*, 2022.
- Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In *International Conference on Learning Representations*, 2023.
- An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pp. 10088–10115, 2023. doi: 10.52202/075280-0441.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyang Luo. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 400–410, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-demos.38.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, pp. 53728–53741, 2023. doi: 10.52202/075280-2338.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626. Association for Computing Machinery, 2023. doi: 10.1145/3600006.3613165.

A Data Construction and Schema

This appendix expands the frozen-graph construction and data conventions described in Section 4.2.

Release selection. The loader streams yearly compressed JSON Lines files, skips malformed rows, flattens the required OCDS fields (Open Contracting Partnership, 2020), and retains the latest parseable release per OCID. It also stores the years in which the process was observed. This creates a current snapshot rather than a merged event history.

Entity resolution. Registry identifiers have priority. Platform identifiers are linked only under the same OCID, normalised name, and role, and only if all observations identify one official entity. The second phase uses government lookup, exact name and region, then exact name. Empty normalised names remain singletons. Fuzzy similarity creates review candidates only. Reviewed safe edges are applied by a cluster consistency check. The current artifacts contain 204,711 raw aliases and 131,502 canonical organisations; 82,013 remain singletons.

Values and provenance. The amount source precedence is award, linked contract, then tender. Award- and contract-sourced values may be labelled additive; tender ceilings are not. Numeric zero is retained. Currency is stored but not converted. Evidence nodes retain the source field and source document linkage.

Data use and provenance. The source material consists of publicly accessible procurement releases concerning real organisations. The graph uses declared public fields and retains source-document links; it does not infer private attributes. Reported answers are therefore scoped to the frozen snapshot and its recorded provenance.

Table 2: Retained graph snapshot. One contract may have several supplier edges.

Node type	Count	Edge type	Count
Contract	215,221	buyer_of	215,218
Organisation	131,502	supplier_of	334,063
CPV	3,870	categorized_by	164,691
Evidence	535,731	evidence_for	1,326,240

B Reader, Planner, and Repair Interfaces

This appendix specifies the learned-call interfaces and information boundaries that populate the persistent trajectory described in Sections 4.3, 4.4, and 4.6.

The prompt builders and provider schemas are generated by `src/procurement_graph/reasoning/typed_planning.py`. Its audited SHA256, together with runner and configuration hashes, is recorded in `paper/tmlr_v3/artifacts/REPRODUCIBILITY_MANIFEST.md`. Table 3 distinguishes settings stored directly in the run summary from those recovered from the audited runner. The implementation is the canonical source for exact prompt formatting, dynamic schema context, and enum expansion.

The audited API client uses a 60-second timeout and allows three retries after the initial request, with waits of 2, 4, and 6 seconds. Rate limits remain retryable; most other 4xx failures terminate immediately. The runner does not set an explicit output-token limit.

Table 3: Teacher prompt and call contract. “Stored” is present in the harvest summary; “reconstructed” is implied by the audited runner and model-dependent prompt selection.

Call	Model	Output control	Sampling and visibility
Step-1 intent	gpt-5.4-nano (stored)	strict intent-program JSON Schema	temperature 0; question and schema slice; no records or oracle
Step-2 graph	grok-4-1-fast- non-reasoning (stored)	reconstructed lean/optional strict graph schema	temperature 0; at most two struc- tural samples; question, Step-1 out- put, and schema slice
Repair reading	Step-1 model	fixed eight-section text scaffold	called for semantic repair; hidden- answer fields removed
Repair graph	Step-2 model	strict repair/graph JSON Schema	at most two teacher-harvest repairs; failed graph and structured feedback, but no oracle value

B.1 Step-1 understander

The exact system instruction is:

“You are the typed intent programmer for a UK public-procurement KGQA system. Convert the question into a small computation program. Do not answer the question. Use only information explicitly present in the question. The output shape is enforced by a strict JSON schema.”

The user JSON contains the keys `question`, `task`, `allowed_slots`, `operation_guidance`, `hard_rules`, and `retrieved_schema_context`. The required output contains:

```
answer_signature: operation, value_type, answer_field_text
program: ordered typed steps with literal filters and dependencies
answer_step: id of the final step
unsupported_or_ambiguous: explicit reason or empty
```

Allowed literal slots are publication year/range, CPV code and label, procurement category limited to goods/services/works, buyer, supplier, exact quoted title, date, and numeric amount. Root steps cannot invent an “all records” input. Later steps may reference only earlier identifiers. Bridge intents must construct source records, a distinct entity set, a dependent target set, and a final operation. Future years remain executable no-result queries rather than predicted abstentions. If the question is unsupported or ambiguous, the official program is empty.

B.2 Step-2 graph planner

Step 2 is the learned graph-planning call used when the builder requires model-based program construction. The teacher model name selects the lean prompt rather than the larger capability-card prompt. Its exact system instruction is:

“You are the structured graph planner for a UK public-procurement KGQA system. Turn the question and the Step-1 briefing into an executable **graph_plan** (variables + return). A strict schema fixes the output shape; fill the fields faithfully and never answer the question.”

It receives the original question, the Step-1 briefing, schema context, and instruction list. Its JSON graph contains a flat variable array, relations, a return record, a template label, and an optional abstention reason. Record filters contain only literal constraints from the question. Derived phrases are represented as variables and relations, not copied into filter slots. A bridge must emit a typed entity set and bind it into the target record variable. Money sums must request the additive-value guard. Unused schema fields take declared empty values rather than invented semantics.

The planner never receives records, an execution result, or the oracle answer. The consistency checker compares the graph with explicit years, CPV codes, categories, titles, numeric literals, buyer/supplier roles, and the requested count target. A structural resample appends only the diagnostic that compilation or consistency failed and asks for a different, simpler decomposition.

B.3 Feedback-conditioned repair

An eligible runtime failure produces a structured record containing its stage, code, path, and message. Repair reading is instructed to revise the understanding from the retained failed state and feedback without answering or inferring a reference answer. Graph repair is instructed to diagnose the failed graph, choose an allowed action, and emit a graph under the original schema. The feedback sanitizer removes gold/reference/expected/oracle-answer keys. Repair SFT conditions on the original question, Step-1 briefing, failed graph, and this visible feedback, and targets the accepted revised graph. Clean no-result, unsupported, and ambiguous cases terminate rather than being replanned in search of a different outcome. The teacher collection budget is two repairs, while development and primary evaluation use one.

B.4 Training examples and information boundaries

Step-1 SFT serialises the question and schema context to the verified interpretation u^+ . Direct-plan SFT serialises $(x, u) \mapsto z^+$. Repair SFT serialises $(x, u, z^-, f) \mapsto z^+$. Preference training uses the same question and briefing with accepted and rejected graph completions. Neither the oracle answer nor the candidate’s returned value is included in the student input or target. The student therefore learns the selected planning decisions and responses to visible failure feedback from the retained trajectory, while reference answers remain outside its inputs and targets.

Offline selection nevertheless uses more than runtime grounding. A direct answerable target must pass execution/release checks, match the hidden answer oracle, and have the expected output shape. A runtime-passing but answer-wrong graph enters the hard-negative path. Its repair becomes a repair-SFT or DPO positive only after deterministic re-execution satisfies the offline oracle and output-shape criteria. Grounding therefore defines the executable boundary, while the oracle defines the denotational selection boundary.

C Full Procurement Runtime Specification

This appendix expands the online program-construction procedure, intermediate representations, and stage-specific checks described in Sections 4.1 and 4.6.

This appendix makes the main, two-stage procurement system explicit before describing the single-call algebra used by PACS and WTQ. The two representations share a deterministic backend, but they are not interchangeable and are never combined within one reported prediction.

C.1 Intermediate representations

Step 1 returns an *intent program*. Its answer signature declares the operation, value type, and requested answer field; its ordered steps declare literal filters, dependencies, and the final answer step. The validated intent is retained as the first learned state in the trajectory. An unsupported or ambiguous item instead carries a reason and an empty official program. The builder then converts a valid intent into the common graph-plan representation; Step 2 supplies the learned construction when required.

Step 2 returns the graph-plan interface in Table 4. This representation is used for a multi-hop dependency such as records $\rightarrow$ emitted suppliers $\rightarrow$ records bound by those suppliers $\rightarrow$ a scalar or set answer. The compiler normalises supported planner idioms, constructs a directed acyclic graph, and lowers every executable variable to a backend query specification.

Table 4: Graph-plan intermediate representation used by the full procurement runtime.

Object	Required content	Deterministic checks
Variable	identifier, kind, semantic role, literal constraints, optional emitted field	unique identifier; legal field and value type; no invented root input
Dependency	upstream variable and downstream consumer	reference exists; topological order; no cycle
Relation	source variable/field and target variable/bind field	emitted and bound types agree; no unconstrained bind
Return	input variable, operation, answer field/type, optional grouping or comparator	operation accepts the variable kind; requested count target and direction agree with the question
Guards	exhaustive reduction, deduplication key, additivity and optional population conditions	safe aggregate and declared backend semantics
Abstention	unsupported or ambiguous reason	no answering graph may coexist with a terminal non-answer decision

C.2 Stage contracts.

Table 5 expands the four contract classes introduced in Section 4.6. The structural class is applied at both the structured-understanding and program-construction boundaries, so the five runtime stages are covered by four contract classes. Each decision and diagnostic is stored with the state at which the check becomes applicable, creating the stage-linked provenance used by repair and later curation. Runtime checks do not receive the reference answer.

C.3 Online algorithm

For one question, the implemented runtime performs the following steps.

1. Select a schema slice from question keywords. This is rule-based schema selection, not dense retrieval over records.
2. Ask the Step-1 model for an intent program at temperature zero. Validate references, dependency order, return type, answer signature, and any terminal non-answer reason.
3. Construct the executable graph under the shared typed interface. Supported intent patterns may be lowered by the builder without an additional model call; otherwise the Step-2 typed graph planner constructs the graph. A repairable Step-1 validation failure follows the repair route instead. In the evaluation protocol, at most two structural samples are considered, and a sample is retried only after a detectable structural failure.
4. Ground aliases, fields, entity literals, emitted values, downstream bindings, aggregation guards, and the deduplication key. Run schema, conflict, and exhaustive-reduction preflight.
5. Execute graph variables in dependency order. Derived value sets become hidden membership constraints on downstream record variables. Compute the declared return operation from the resulting record population.
6. Run postflight, answer-sanity, and evidence-release checks. Return an answer card only when the resulting state is releasable. A clean no-result, unsupported, or ambiguous state is final.
7. If a structured-understanding, compilation, grounding, execution, or named release failure is explicitly repairable, supply the structured failure to the repair procedure and repeat the affected downstream stages. Main evaluation permits one feedback revision; teacher collection permits two.

Table 5: Paper-level predicate inventory for the deployed stage contracts. Diagnostic entries are the human-readable families used in the audit; exact code-level checks are bound to the audited source recorded in Appendix J. Runtime checks do not receive the reference answer.

Boundary	Checked object	Predicate family	Diagnostic family	Repair
Understand	u	intent schema; reference and dependency validity; answer signature; terminal non-answer form	structural interpretation failure	Yes
Understand	(q, u)	question-supported literals and retention of schema-mapped conditions	missing or unsupported mapped condition	Yes
Program	z	graph schema; variable identifiers; acyclic dependencies; operator signatures and required arguments; return type and consistency with u	invalid graph plan	Yes
Ground	z_K	field/value domains; schema membership; entity-resolution confidence; buyer/supplier role binding	no confident schema match; other grounding rejection	Yes
Ground	(z, z_K)	declared answer/value types and operation input/output compatibility after lowering and binding	type contract violated; type contract rejected	Yes
Ground	(u, z_K)	retention of mapped question atoms and consistency of the grounded operation with the interpreted request	compiled plan dropped an atom; operation contract violated	Yes
Ground	z_K	exhaustive population construction; deduplication key; monetary additivity guard	other grounding rejection / unsafe aggregate	Yes
Execute	r	successful deterministic operator evaluation	execution error	Yes
Execute	r	scalar cardinality when a unique scalar is required	multiple answers for a scalar	Yes
Execute	r	empty-result state under the declared outcome semantics	no results	Only when not a clean empty
Release	$(r, \hat{y})$	postflight status; answer shape; return consistency; releasable evidence	shape, consistency, evidence, or release failure	Named failures only

The online path never receives a gold answer. Structural sampling and repair are therefore driven only by represented runtime failures; semantic correctness remains an offline evaluation criterion.

C.4 Evidence and release contract

Table 6 distinguishes the state retained during runtime execution from what is persisted in each evaluation artifact. It also separates exact execution state from bounded audit material, preventing an identifier sample from being mistaken for the complete population used by a count or sum.

D Recursive Algebra

This appendix gives the operator-level details for the executable representation introduced in Sections 4.4 and 4.5.

The direct-composition language is a closed tree algebra. Programs are limited to depth 16 and 64 nodes. Table 7 gives the public node signatures used by procurement/PACS and the single WTQ-specific extension, **extreme_rows**; predicates include equality, membership, substring containment, existence, numeric bounds, negation, disjunction, and membership in a computed **Values** set.

Counts and grouped counts deduplicate contract identifiers. Projections remove null and empty values and return sorted distinct strings. Sums use decimal accumulation. Ranking ties are ordered by their stored string keys. A Boolean matches only a Boolean, so **True** is not equal to numeric one. The type checker does not by itself enforce every domain convention: additivity, schema membership, and currency remain separate contracts.

Table 6: Information produced and retained by the procurement runtime.

Quantity	Runtime semantics	Retention boundary
Answer and population count	exact deterministic result under the frozen record view	stored in a rich trace; prediction and correctness are retained by the slim main evaluator
Variable trace	grounded constraints, dependencies, emitted field, status, and output size	available when rich tracing is enabled; omitted from the main 2,285-row result files
Contract evidence	matching contract identifiers and rows	exact total count, but identifiers/rows are capped at 200 per execution result
Public evidence view	operation, answer, count, identifiers, and OCIDs	serialises at most 50 identifiers and 50 OCIDs from the runtime bundle
Source documents	URL and document metadata extracted from OCDS	retained for provenance; passage-level scoring is outside this evaluation
Oracle	typed answer used for offline curation and scoring	never included in online prompts or student targets

Table 7: Recursive-algebra operators. The procurement/PACS core is reused on WTQ, which adds the row-preserving `extreme_rows` node. The validator checks compositional types and required arguments before execution.

Node	Signature	Function
<code>filter</code>	<code>predicates → Records</code>	filter records
<code>values</code>	<code>Records → Values</code>	project distinct values
<code>count, sum</code>	<code>Records → Number</code>	aggregate records
<code>size</code>	<code>Values → Number</code>	set cardinality
<code>exists</code>	<code>Records → Bool</code>	test non-emptiness
<code>select, extreme</code>	<code>Records → Value</code>	scalar or extremal lookup
<code>groupby</code>	<code>Records → Groups</code>	grouped count or sum
<code>argext, top</code>	<code>Groups → scalar/ranking</code>	rank groups
<code>combine</code>	<code>Number² → scalar</code>	arithmetic or comparison
<code>vcompare</code>	<code>Value² → Bool</code>	compare values
<code>setop</code>	<code>Values² → Values</code>	union, intersection, difference
<code>gcombine</code>	<code>Groups² → Groups</code>	combine aligned groups
<code>keys_where</code>	<code>Groups → Values</code>	threshold group keys
<code>extreme_rows</code>	<code>Records → Records</code>	row-preserving arg-extremum (WTQ)

E Benchmark Construction and Integrity

This appendix expands the benchmark-construction and split-integrity procedure summarised in Section 4.7.

The initial procurement generator creates executable answer specifications from the graph. Gate A checks conflicts, identifier integrity, deterministic execution, and aggregate sanity before surface generation. A second language-model stage writes the question; Gate B checks that visible filters and operations agree with the specification. The initial artifacts contain 10,000 answer specifications, 9,053 generated questions, and 9 Gate-B failures, yielding 9,044 accepted base question–program pairs. Gate B is the automated specification-consistency screen used at this construction stage. Benchmark expansion is then non-one-to-one: answerability twins and additional surface realisations derive multiple rows from selected base programs; whole-program grouping keeps all descendants of a program in one split; and class balancing determines the retained mixture. The net expansion is 3,784 rows, yielding 12,828 benchmark rows: 9,267 training, 671 development-select, 556 development-tune, 49 smoke, and 2,285 frozen-evaluation rows. The 671 development-select and 556 development-tune rows were reserved during construction but are not used in the reported experiments. The reported checkpoint-comparison pipeline instead designates a balanced 260-question subset of the frozen evaluation partition for model selection before any primary testing; the remaining 2,025 rows are never used for selection.

Second-evaluator audit. In a separate oracle-audit pool, the benchmark oracle and a second operation implementation agree on 14,752 of 14,770 paired answerable evaluations. The second implementation uses separate operation logic while sharing the same upstream OCDS transformation, entity-resolution output, frozen Parquet files, and declared data conventions. The residual 18 cases comprise 15 top- k parameter inferences and 3 Boolean surface parses. The reported 99.88% is therefore interpreted as an implementation cross-check under shared data conventions.

Table 8: Composition of the 2,285-row frozen evaluation partition. The 260-row development subset contains 20 rows from each bucket and is held out by identifier before evaluation on the remaining 2,025-row primary set.

Bucket	Rows	Bucket	Rows	Bucket	Rows
Ambiguous	120	Boolean	121	Bridge join	350
No result	120	Categorical	120	Comparison	306
Unsupported	120	Count	250	Factoid	250
Min/max	107	Set	93	Sum	200
Top- k	128	Answerable total	1,925	Grand total	2,285

F Training Data and Configurations

This appendix records the supervision pools and optimisation settings underlying Sections 4.8 and 5.3.

Table 9 separates verified source pools from the objective-specific rows actually exported for optimisation. Direct-plan examples are down-selected after verification with benchmark bucket as the stratification variable, preventing high-frequency question clusters from dominating Step 2 SFT. The resulting stored export contains 2,787 training and 57 validation rows from the 5,606-row eligible pool; the reported models consume these fixed exported rows directly rather than re-sampling the eligible pool. Abstention targets are subject to the separate ten-percent cap shown in Table 9. Overlapping objectives do not create new unique questions.

Table 9: Teacher trajectory pools and exported training splits. Verified pools and exported optimisation rows are reported separately.

Artifact	Verified / candidate rows	Exported train/validation
Training questions	9,267	–
Answer-producing trajectories passing runtime contracts	6,860	–
Oracle-verified direct-plan trajectories	5,606	–
Runtime-accepted, oracle-wrong trajectories	1,254	–
Oracle-correct repaired trajectories	1,747	–
Repair SFT export	–	1,679 / 46
Teacher preference pairs	390	390 / –
Abstention target pool	590	included subject to a ten percent cap
Final plan SFT export	–	2,787 / 57
Step-1 intent targets	–	5,091 / 96

Table 10 records the configurations actually consumed by the reported Qwen3-8B and Llama-3.1-8B checkpoints (Yang et al., 2025; Grattafiori et al., 2024). Every row uses 4-bit QLoRA on all target modules, rank 64, $\alpha = 128$, dropout 0.05, bfloat16 compute, effective batch 16, cosine decay, and one CUDA process (Dettmers et al., 2023; Zheng et al., 2024). Step-2 SFT trains on 4,466 rows comprising 2,787 direct-plan rows and 1,679 repair rows. The configured model-selection split contains 57 direct-plan rows, while 46 repair rows are retained as a separate validation export. The preference checkpoints use the sigmoid DPO objective (Rafailov et al., 2023).

Table 10: Training and continuation configurations. Q/L denotes matched Qwen3-8B and Llama-3.1-8B variants.

Stage		Initialisation and train rows	Objective	Epochs	LR	Cutoff / validation
Step-2 (Q/L)	SFT	base; 2,787 plan + 1,679 re-pair	token SFT	3	10^{-4}	6,144 / 57 plan
Step-1 (Q/L)	SFT	base; 5,091 intent programs	token SFT	3	10^{-4}	6,144 / 96
Qwen RSFT		Qwen SFT; 3,019 plan + 3,169 repair	token SFT	2	5×10^{-5}	6,144 / 65 plan
Llama RSFT		Llama SFT; 3,026 plan + 3,190 repair	token SFT	2	5×10^{-5}	6,144 / 66 plan
Qwen DPO		Qwen RSFT; 390 teacher + 689 on-policy pairs	sigmoid DPO, $\beta = .1$	1	5×10^{-6}	6,144 / none
Llama DPO		Llama RSFT; 390 teacher + 693 on-policy pairs	sigmoid DPO, $\beta = .1$	1	5×10^{-6}	6,144 / none
Composition model		fresh Qwen; 12,414 trees	token SFT	2	10^{-4}	4,096 / 253
WTQ A		fresh Qwen; 4,615 answer-matched trees	token SFT	2	10^{-4}	4,096 / 94
WTQ C		fresh Qwen; about 5,423 translated programs	token SFT	2	10^{-4}	4,096 / –

Training logs record Transformers 5.6.0 and the expected example and optimiser-step counts. The evaluation server logs record vLLM 0.24.0 (Kwon et al., 2023), seed 0, bfloat16, maximum sequence length 8,192, LoRA rank 64, and disabled Qwen thinking. Reported local training and evaluation use one CUDA process; the archived software and runtime configuration define the reported setup.

G PACS Details

This appendix gives the PACS protocol supporting the paraphrase-robustness results in Section 5.4.

PACS instantiates seven families at levels L1–L3. Construction samples executable typed trees, runs the production and separate evaluators, requires answer agreement and non-degeneracy, and excludes exact tree hashes found in composition-model training or validation. Missing-operator and no-result controls are added separately. Channels A and B share the same executable tree, oracle, and planner and differ only in surface rendering. The evaluated surface files contain 922 rows. Table 11 reports the status and exposure labels used by evaluation.

Table 11: PACS evaluation composition. “Seen” refers to the declared program-shape label, not exact tree overlap.

Status	Rows	Shape label	Rows
Answerable	730	Seen	431
Unsupported	138	Unseen	491
No result	54	Total	922

Reported PACS scores use the v1.1 evaluation manifest and deterministic re-execution of the saved trees, yielding 722/922 for channel A and 711/922 for channel B. Exact question and tree-hash overlap with composition-model training or validation is excluded. The declared seen/unseen labels are retained as composition-difficulty metadata; the primary A/B comparison measures paraphrase robustness under a shared executable tree and oracle.

H WikiTableQuestions Evaluation

This appendix gives the WTQ protocol supporting the cross-domain reuse results in Section 5.4.

The WTQ evaluation uses 4,344 questions over 421 test tables absent from the training-table partition. Each table is exposed through two aligned views. The typed view adds deterministic numeric, date, year, month, and normalised-string columns for computation; the raw view preserves the strings returned to the official scorer. Row identifiers and derived columns are never shown as answer values. The procurement/PACS algebra is otherwise retained, with one additional row-preserving **extreme_rows** node for row-level arg-extremum questions.

For official scoring, each predicted answer item first replaces tab and newline characters with a space. The question identifier and cleaned items are then joined with tab delimiters and passed to **scripts/wtq/official_evaluator.py** version 1.0.2 with the original CoreNLP-tagged targets. Table 12 records the resulting official counts and paired transitions.

Table 12: WTQ official evaluation. Gain/loss compares each row with the row immediately above using exact McNemar tests.

Planner or supervision	Correct / 4,344	Accuracy (%)	Gain/loss; p
Untuned Qwen3-8B	978	22.51	–
Procurement-trained Qwen	1,187	27.33	387/178; 8.50×10^{-19}
Answer-only self-harvest (A)	1,930	44.43	833/90; 1.77×10^{-151}
Translated gold programs (C)	2,250	51.80	628/308; 6.20×10^{-26}

The A supervision pool contains 4,709 answer-matched examples, split into 4,615 training and 94 validation examples. The C run uses approximately 5,423 translated program examples selected by translation, type, execution, and answer agreement.

I Evaluation Protocol Matrix

This appendix consolidates the evaluation boundaries summarised in Section 4.8.

Table 13 records the decoder and authority boundaries needed to interpret the reported protocols. A dash means that the component does not exist in that protocol, rather than that it failed to run.

The cloud reference on the 260-question development set is evaluated three times and reported as mean and standard deviation. On the primary set, each frozen configuration is run three times on the same 2,025 questions. The run-to-run SDs are 0.96 pp (cloud), 1.38 pp (local Qwen), 0.51 pp (local Llama), 1.17 pp (hybrid Qwen), 0.56 pp (hybrid Llama), 0.58 pp (top-40 retrieval), 0.61 pp (gold-constraint retrieval), and 0.74 pp (closed book). Repeating the fixed question set measures inference variability rather than test-sample or training-seed variance. Coverage, release outcomes, and paired transition counts use the archived per-item run retained for item-level analysis, and the McNemar comparison uses the corresponding archived local-Qwen and cloud predictions reported in Section 5.1.

J Artifact and Reproduction Notes

This appendix records the implementation and artifact provenance supporting the reproducibility details in Section 4.8.

The implementation entry points are:

- `pipelines/01_ingest.py` through `pipelines/04_extract.py`, followed by `pipelines/40_build_kg.py` and `pipelines/41_validate_kg.py`;
- `scripts/apply_safe_er_merges.py` for the reviewed cluster-safe entity-resolution layer;

Table 13: Inference and curation protocols. “Oracle” refers to access during the named procedure, not to whether an oracle later exists for scoring.

Protocol		Plan form	Decoding	Repair	Oracle and saved output
Teacher (9,267)	harvest	Step-1 intent; conditional Step-2 graph	temperature 0; two structural samples	at most 2	oracle-hidden online; typed oracle/shape filter starts only after run termination; rich typed trajectory and separate training sinks
Procurement (2,285; 2,025)	main primary	same full runtime	temperature 0; two structural samples	at most 1	no online oracle; saved evaluator files retain slim per-item predictions
Development (260)		same full runtime; checkpoint varies	temperature 0; two structural samples	at most 1	no online oracle; paired predictions by question
PACS (922)		one recursive algebra tree	temperature 0; primary protocol is prompt-only JSON	none	no online oracle; tree, execution status, answer, and offline score saved
WTQ (4,344)		one recursive algebra tree with per-table schema	temperature 0; JSON-schema guided	none in the reported run	no online oracle; answer items rescored by the official evaluator

- `scripts/run_teacher.py` and `scripts/export_llamafactory.py` for graph-plan curation and export;
- `scripts/run_compare.py` for the full runtime, and `scripts/run_compose_probe_eval.py` for direct composition.

Paper-side derived counts and provenance notes are stored in `paper/tmlr_v3/artifacts/number_ledger.json`. The ledger assigns each reported value a scope, metric, artifact source, and derivation, while generated files under `paper/tmlr_v3/generated` supply recurring tables and numeric claims. The corresponding input, runner, and result hashes are recorded in the reproducibility manifest.

All reported correctness values use the same strict type-aware scorer. The number ledger records the metric definition, scope, derivation, and source artifact for recurring numerical claims so that headline and appendix results are tied to a single scoring definition.

The prompt/call contract, source and configuration hashes, selection authority, serving settings, and comparison scope are recorded in `paper/tmlr_v3/artifacts/REPRODUCIBILITY_MANIFEST.md`. Settings recovered from the audited runner rather than stored directly in a run summary are marked as reconstructed. The paper is therefore bound to the enumerated source, data, configuration, checkpoint, decoder, and scorer artifacts recorded in the manifest.

K Comparison Scope

This appendix clarifies the scope of the comparisons reported in Sections 5.1 and 5.4.

Table 14 records the scope of each controlled or comparative analysis. The comparisons answer different questions and are interpreted according to what is held fixed.

These comparisons address distinct questions: checkpoint selection, system configuration, evidence access, paraphrase robustness, and cross-domain reuse. They are interpreted according to the factors held fixed rather than as a single ablation chain, consistent with the scope stated in Section 6.

Table 14: What each existing comparison holds fixed and what it can establish.

Evidence	Held fixed	Changed	Interpretation
Checkpoint ladder (260)	questions, scorer, runtime budget, base within family	training stage and objective	model-selection progression within each family
Hybrid/fully local runtime	questions and downstream deterministic runtime	hybrid changes Step 2; fully local changes both learned stages	stage-wise system configuration comparison
Access controls	answering model and evaluation questions	available evidence and retrieval access	effect of bounded evidence access versus graph execution
PACS A/B	program, oracle, and planner	surface rendering	paraphrase robustness
WTQ base/v3/A/C	test set and official scorer	source and amount of supervision	cross-domain reuse and supervision comparison

L Runtime Repair Audit

This appendix provides the detailed repair counts underlying Section 5.2.

Table 15 reconstructs the final outcomes of answerable teacher-harvest attempts that triggered at least one formal repair. The initial state is a runtime failure rather than a scored answer, so the denominator represents formal repair triggers rather than completed wrong answers.

Table 15: Teacher harvest outcomes after bounded repair. Runtime acceptance and strict oracle correctness are evaluated separately.

Outcome	Count
Answerable attempts triggering repair	3,626
Total repair attempts	3,935
Oracle-correct after repair	1,747
Runtime-accepted, oracle-wrong	313
Unresolved	1,566
Runtime accepted after repair	2,060
Oracle-correct rate	48.18%

For the 1,747 oracle-correct repaired trajectories, the retained repair records are categorised as 1,604 plan-compilation, 125 executor-check, 17 schema-validation, and 1 graph-execution cases. This categorisation describes the origin of successful repair records and is distinct from the contract-activation frequencies reported in Section 5.2.

Illustrative repair and authority cases. The retained audit also contains examples of three qualitatively different outcomes. In one money-valued minimum query, grounding rejects a plan that returns a record rather than money (`answer_returns_record_not_money`); typed feedback leads to a replanned arg-min over the value field, after which grounding, execution, and verification pass. In a buyer-identification query, execution returns `multiple_answers`; two teacher-harvest repair attempts fail to produce a unique answer and the run abstains when the repair budget is exhausted. A third query passes the represented runtime checks but returns a source-record count of 37 against an offline oracle value of 6,962 because the intended relational hop is absent from the plan. The three cases respectively illustrate successful local repair, budget-bounded unresolved failure, and the runtime-accepted/oracle-wrong boundary quantified in Section 5.3.